Prob 1. (30 pts) Assume we have the following memory

Address	Byte 0	Byte 1	Byte 2	Byte 3
0x2000_0000:	0x88	0x89	0x8A	0x8B
0x2000_0004:	0x8C	0x8D	0x8E	0x8F
0x2000_0008:	0x70	0x71	0x72	0x73
0x2000_000C:	0x74	0x75	0x76	0x77

Also assume we have the following assembly function code snippets:

```
task1:                              |   task2:
        ldrsh r2, [r0, #4]          |           ldrsb r2, [r0, #2]!
        str   r2, [r1], #4          |           str   r2, [r1, #0]
        ldrsh r3, [r0, #8]          |           ldrsb r3, [r0, #4]!
        str   r3, [r1], #4          |           str   r3, [r1, #4]
        bx    lr                    |           bx    lr
```

which are called in C with the following code:

```
void *p1 = (void *)0x20000000;
void *p2 = (void *)0x20000020;
void *p3 = (void *)0x20000030;
task1(p1, p2);
task2(p1, p3);
```

1. (8 pts, 4 each) What are the values of R2 and R3 in hexadecimal just before returning from task1 (assume we have a breakpoint at `bx lr`)?
2. (8 pts, 4 each) Repeat the above for task2.
3. (8 pts) Draw the memory map for 8 bytes starting at 0x2000_0020 as shown in the form given above.
4. (6 pts) Assume R0 holds `ptr`, a pointer of `uint16_t` number, and R1 holds `i`, an integer. Write the ASM code for `val = *(ptr + i)`, where `val` is R2.

Solution:

Part 1.

- `ldrsh r2, [r0, #4]` ==> r2 = 0xFFFF_8D8C.
- `ldrsh r3, [r0, #8]` ==> r3 = 0x0000_7170.

Part 2.

- `ldrsb r2, [r0, #2]!` ==> r2 = 0xFFFF_FF8A. (r0 = 0x2000_0002)
- `ldrsb r3, [r0, #4]!` ==> r3 = 0xFFFF_FF8E.

Part 3.

We need to consider the store operations in `task1`:

Address	Byte 0	Byte 1	Byte 2	Byte 3
0x2000_0020:	0x8C	0x8D	0xFF	0xFF
0x2000_0024:	0x70	0x71	0x00	0x00

Part 4.

```
LDRH R2, [R0, R1, LSL #1]
```

> -:-

Prob 2. (30 pts) Assume we have the following memory map:

Address	Byte 0	Byte 1	Byte 2	Byte 3
0x2000_0000:	0x7C	0x7D	0x7E	0x7F
0x2000_0004:	0x80	0x81	0x82	0x83
0x2000_0008:	0x84	0x85	0x86	0x87
0x2000_000C:	0x88	0x89	0x8A	0x8B

(20 pts). Translate the following C code into assembly assuming `int16_t *src = 0x2000_0000` is saved in R0 and `int16_t *dst = 0x2000_0008` in R1:

```
*dst++ = *src++;
*dst++ = *++src;
*dst++ = ++*src;
```

(10 pts). Determine the values in the above memory after the above operations, which are executed one by one.

Part 1:

```
@ (1) *dst++ = *src++;
    LDRSH R2, [R0], #2
    STRH  R2, [R1], #2

@ (2) *dst++ = *++src;
    LDRSH R2, [R0, #2]!
    STRH  R2, [R1], #2

@ (3) *dst++ = ++*src;
    LDRSH R2, [R0]
    ADD   R2, #1
    STRH  R2, [R0]
    STRH  R2, [R1], #2
```

Part 2:

Address	Byte 0	Byte 1	Byte 2	Byte 3
0x2000_0000:	0x7C	0x7D	0x7E	0x7F
0x2000_0004:	0x81	0x81	0x82	0x83
0x2000_0008:	0x7C	0x7D	0x80	0x81
0x2000_000C:	0x81	0x81	0x8A	0x8B

Prob 3. (20 pts) Assume we are given code snippets below:

Part A: C code:

```
// Functions defined in a .s file
extern void mp_task(uint8_t *pIpt, int32_t *pOup);
uint8_t ipt[16];    // Input data
int32_t opt[4];     // Output data
int main(void) {
    for (int i = 0; i < 16; i++) {
        ipt[i] = i<<4;
    }
    mp_task(ipt, opt);
    printf("Out1 = 0x%X, Out2 = 0x%X, ", opt[0], opt[1]);
    printf("Out3 = 0x%X, Out4 = 0x%X\n", opt[2], opt[3]);
    while (1);
}
```

Part B: assembly code:

```
mp_task:
    ldrd    r2, r3, [r0, #8]!
    str     r3, [r1, #0]
    str     r2, [r1, #4]
    sub     r0, r0, #8
    ldrsh   r3, [r0], #4
    ldrsh   r2, [r0, #4]!
    strd    r2, r3, [r1, #8]!
    bx      lr
```

(12 pts) Determine the printout of Task 1.

(8 pts) Assume the addresses of `ipt` and `opt` are `0x2000_0200` and `0x2000_0280`, respectively. Determine the values `r0` and `r1` in hexadecimal just before existing `mp_task`.

Soln:

The memory map of the problem is given as

Address	Byte 0	Byte 1	Byte 2	Byte 3
ipt + 0x00:	0x00	0x10	0x20	0x30
ipt + 0x04:	0x40	0x50	0x60	0x70
ipt + 0x08:	0x80	0x90	0xA0	0xB0
ipt + 0x0C:	0xC0	0xD0	0xE0	0xF0

```
task1:
    ldrd    r2, r3, [r0, #8]! @ R2 = 0xB0A0_9080, R3 = 0xF0E0_D0C0
                              @ R0 = 0x2000_1018
    str     r3, [r1, #0]      @ 0x2000_1040: 0xF0E0_D0C0
    str     r2, [r1, #4]      @ 0x2000_1044: 0xB0A0_9080
    sub     r0, r0, #8        @ R0 = 0x2000_1010
```

```
7       ldrsh   r3, [r0], #4       @ R3 = 0x0000_1000, R0 = 0x2000_1014
8       ldrsh   r2, [r0, #4]!      @ R2 = 0xFFFF_9080, R0 = 0x2000_1018
9       strd    r2, r3, [r1, #8]! @ 0x2000_1048: 0xFFFF_FF80; R1 = R1 + 8
10      bx      lr                @ 0x2000_104C: 0x0000_0000
```

Part (a).

```
1  Out1 = 0xF0E0_D0C0, Out2 = 0xB0A0_9080
2
3  Out3 = 0xFFFF_9080, Out4 = 0x0000_1000
4
5  r0 = 0x2000_0208, r1 = 0x2000_0288
```

-:-

Prob 4. (25 pts) Given the following C function for returning a 32-bit value using a pointer:

```c
int load_based_on_x_c(int x, int *ptrA, int *ptrB) {
    if (x <= -20 || x >= 20) {
        return (int32_t)*(ptrA+1) * 4;
    } else {
        return *(ptrB+2) / 4;
    }
}
```

Translate the above C function into ASM with the function name being `load_based_on_x_s` using the conditional branch method.

Solution:

```
@ int load_based_on_x_s(int x, int16 *ptrA, int *ptrB)
@ x -> r0, *ptrA -> r1, *ptrB -> r2
load_based_on_x_s:
    cmp     r0, #-20
    ble     load_based_on_x_s_then
    cmp     r0, #20
    blt     load_based_on_x_s_else
load_based_on_x_s_then:
    ldr     r0, [r1, #4]
    lsl     r0, #2
    b       load_based_on_x_s_end
load_based_on_x_s_else:
    ldr     r0, [r2, #8]
    lsr     r0, #2
load_based_on_x_s_end:
    bx      lr
```

-:-